INSTITUTO FEDERAL
Rio Grande do Sul | Campus Porto Alegre

Curso: SSI - Sistemas para Internet
Disciplina: Programação para Web II
Professor: Rodrigo Prestes Machado
Estudante: José Athaualpa Kolesny Tricot

Proposta de Trabalho Final

BiciRotas - Sistema de Mapeamento e Condições de Rotas de Ciclismo

1. Conceito do Sistema

O **BiciRotas** é um sistema de retaguarda (backend) voltado para ciclistas que exploram estradas rurais e trilhas. O objetivo principal do usuário final é consultar rotas cadastradas e verificar as condições atuais daquele trajeto — por exemplo, se a estrada apresenta muita lama após uma chuva, se está bloqueada, ou a nota geral de qualidade do percurso.

O contexto de uso envolve dois domínios principais isolados de forma independente: o catálogo das rotas geográficas em si, e o sistema de avaliações e feedbacks deixados continuamente pela comunidade ciclista.

2. Arquitetura de Micro Serviços

O sistema será arquitetado em dois micro serviços implementados em Java utilizando o framework Quarkus:

- **Micro Serviço de Rotas (`ms-rotas`):** Responsável por gerenciar o catálogo principal de rotas (cadastros contendo nome, distância, cidade e dificuldade).
- **Micro Serviço de Avaliações (`ms-avaliacoes`):** Responsável por receber, persistir e listar os feedbacks e atualizações de condições das rotas.

Comunicação: A integração entre os domínios se dará de forma síncrona. Quando um usuário solicitar os detalhes completos de uma rota no `ms-rotas`, este micro serviço atuará como cliente, fazendo uma requisição HTTP via **REST Client** para o `ms-avaliacoes`. Ele buscará todas as avaliações associadas ao ID da rota consultada e retornará um objeto unificado.



3. Modelo de Dados

A persistência será descentralizada. Cada micro serviço terá seu próprio banco de dados isolado (utilizando H2 Database ou PostgreSQL) gerenciado via JPA/Hibernate.

- **No `ms-rotas` (Entidade Rota):**
  - `id` (Long) - Chave Primária
  - `nome` (String)
  - `cidade` (String)
  - `distanciaKm` (Double)
  - `nivelDificuldade` (String)
- **No `ms-avaliacoes` (Entidade Avaliacao):**
  - `id` (Long) - Chave Primária
  - `rotaId` (Long) - Chave de referência externa ao domínio de rotas
  - `nomeCiclista` (String)
  - `condicaoAtual` (String) - Ex: "Muita lama", "Seco", "Em manutenção"
  - `nota` (Integer) - Escala de 1 a 5

4. Lista de Funcionalidades a Serem Desenvolvidas

Endpoints do `ms-rotas`:

- `POST /rotas`: Cria e persiste uma nova rota no catálogo.
- `GET /rotas`: Lista o catálogo básico com as informações de todas as rotas.
- `GET /rotas/{id}/detalhes`: Busca a rota no banco de dados local e, em seguida, efetua uma chamada via **REST Client** ao `ms-avaliacoes` para recuperar todas as avaliações referentes a este ID, retornando um DTO unificado.

5. Requisitos Não Funcionais Aplicados

Visando entregar um sistema robusto e demonstrar os conceitos abordados na disciplina, os seguintes mecanismos serão implementados:

- **Health Checks:** Implementação da extensão `smallrye-health` para expor os endpoints de Liveness e Readiness, permitindo monitorar a saúde de ambos os micro serviços e suas conexões de banco de dados.
- **Tolerância a Falhas (Fallback):** Uso da extensão de *Fault Tolerance* no REST Client. Caso o `ms-avaliacoes` fique indisponível, o `ms-rotas` utilizará a anotação `@Fallback` para interceptar o erro e retornar os dados da rota com uma lista vazia de avaliações e uma mensagem amigável, garantindo a resiliência do sistema.
- **Observabilidade (Logging):** Uso de logs estruturados em pontos críticos da aplicação, especialmente registrando o fluxo de chamadas do REST Client e eventuais ativações do mecanismo de fallback.

6. Referências

- Material didático e aulas da disciplina de Programação para Web II.
- Documentação oficial do framework Quarkus. Disponível em: *https://quarkus.io/guides/*
- Quarkus REST Client Guide. Disponível em: *https://quarkus.io/guides/rest-client*
- Quarkus Fault Tolerance Guide. Disponível em: *https://quarkus.io/guides/smallrye-fault-tolerance*